School of Engineering, Applied Sciences and Technology

BCS 306– Database Management Systems | FA 2025-26

Final Assignment

Course Code: BCS 306	**Course Name:** Database Management Systems
Deadline: 20 November 2025	**Date:** 22 October 2025 – 20 November 2025
Location: Upload on Moodle	**Instructor(s):** Dr. Sahil Garg

Course Learning Outcomes		**PLOs**
CLO-1	Explain fundamental DBMS concepts and architecture for effective data management.	7
CLO-2	Apply relational model concepts and relational algebra to define, manipulate, and query relational schemas.	2
CLO-3	Apply ER modeling and normalization techniques to design and evaluate database schemas.	2
CLO-4	Develop basic and advanced SQL queries to define structures and retrieve data using joins, subqueries, and aggregations.	2
CLO-5	Apply JDBC, indexing, and security techniques to develop efficient and secure database applications.	6
CLO-6	Exhibit effective writing and presentation skills in a database teamwork assignment.	3,5

CLO Mappings

Part 1 (6 Marks)	**Part 2 (4 Marks)**	**Part 3 (5 Marks)**	**Total**
CLO 2, CLO 3, CLO 4	**CLO 6**	**CLO 6**	**15 Marks**

Note: Grade will be re-scaled to match the exact weight of each part

Student ID	**Student Name**	**Part 1**			**Part 2**	**Part 4**	**Total (15)**
		Part 1A CLO 2 (2)	**Part 1B CLO 3 (2)**	**Part 1C CLO 4 (2)**	**CLO 6 (4)**	**CLO 6 (5)**	
20210001983	Leanne Jessica Rodrigo						
20220002458	Aysha Ejaz						
20230003798	Yasmin Issa						

Group Assignment Description and Requirements

You are asked to implement the below tasks and all the required documents (as asked) including the attachments of the implementation source code and program output screenshots.

Submission Instructions

- **This is a group-based assignment.** Each group must have **a minimum of 2 and a maximum of 3 members**. **You are only allowed to form groups with students from your own lab session. Cross-lab groups are not permitted.**
- The assignment consists of **three parts: Implementation, Reporting part, and Q&A session. There will be one common grade for the entire group.**
    - ***Implementation Part* -** This part has three subcomponents, each aligned with specific Course Learning Outcomes (CLOs):
        - Database Design and Implementation - evaluates CLO 2
        - Quality Assessment - Normalization Analysis - evaluates CLO 3
        - Analytical Queries - evaluates CLO 4

      The implementation must strictly follow the assignment requirements. Submit the complete source code.
    - **Reporting Part**
        - Includes Team Report and Presentation, which evaluates CLO 6.
        - The report should include the assignment description and screenshots of all outputs.
    - ***Q&A Part***
        - Covers both theoretical and practical aspects of the assignment**.**
        - This session also evaluates CLO 6.

Additional Submission Guidelines

- **Submission Deadline:** The **group report** and **complete source code** must be uploaded by the specified deadline. The submission must be **well-organized**, and the report must clearly include the **names of all team members**.
- **Single Submission Policy: Each group must submit only once**. Multiple submissions will result in an automatic grade deduction.
- **Academic Integrity:** All reports will be checked via Turnitin for similarity and AI-generated content. Plagiarism is a serious offense and will be dealt with as per the Academic Integrity Policy in the Student Handbook.
- **Text-Based Reports:** Reports must be in text format and not submitted as images. Submitting text as images will result in a **zero mark** for the assignment.

- **Printed Reports for Q&A:** Each team must bring a **printed copy** of the report with the team members' names on the cover page.

Grading Rubric

Evaluation Criteria	Low (1-6)	Needs Improvement (7-9)	Good (10-12)	Excellent (13-15)
Communicate ideas in written reports with appropriate computing-related visuals and documentation (10 Marks)	Fails to communicate ideas in writing; content is unclear, incomplete, or inaccurate. Visuals are missing or irrelevant. Demonstrates minimal understanding of relational models, ER modeling, SQL concepts, and lacks evidence of teamwork communication.	Conveys some ideas in writing but lacks clarity, structure, or completeness. Visuals are limited or poorly integrated. Shows partial application of ER design or SQL queries with inconsistent integration of database concepts and teamwork communication.	Conveys ideas clearly and logically in writing; visuals are relevant and generally well-presented; minor lapses in detail or formatting. Demonstrates solid application of relational model concepts, normalization, and SQL querying. Writing and visuals support teamwork objectives but with some minor inconsistencies.	Conveys ideas with exceptional clarity, accuracy, and structure; visuals are precise, professional, and integrated with the text. Applies ER modeling and SQL techniques to enhance communication. Demonstrates outstanding writing skills in a collaborative database assignment.
Q&A / Presentation Performance (5 Marks)	Provides unclear, incorrect, or minimal responses in the Q&A session. Shows little to no understanding of the assignment or underlying database concepts.	Provides some correct answers but shows gaps in understanding, clarity, or depth. Demonstrates a basic understanding of assignment decisions and database principles.	Responds clearly and accurately to most Q&A prompts, showing solid understanding. Occasional inaccuracies or incomplete answers slightly reduce clarity and rating.	Provides insightful, accurate, and clear responses during the Q&A. Shows strong understanding and communicates confidently.

Workshop Booking System Database Assignment

This assignment requires students to design, implement, and analyze a comprehensive database system for managing workshop bookings in an educational environment. The system focuses on group reservation management, automated capacity control, and real-time slot availability tracking using MySQL Workbench as the primary development tool.

Assignment Objectives

- Demonstrate proficiency in relational database design principles
- Apply normalization techniques and entity-relationship modeling
- Implement complex business logic through database constraints and triggers
- Develop advanced SQL query skills for data analysis and reporting
- Master MySQL Workbench for database modeling and implementation

Technical Competencies

- MySQL Workbench proficiency for database design and implementation
- Advanced SQL programming including stored procedures and functions
- Database performance optimization through indexing strategies
- Data validation and constraint implementation
- Report generation and analytical query development

System Overview

The Workshop Booking System serves as a centralized platform for managing educational workshop registrations, specifically focusing on database operations training (CREATE, UPDATE, DELETE operations). The system accommodates both individual and group reservations while maintaining strict capacity controls and providing real-time administrative oversight.

Core System Capabilities

1. **Group Reservation Framework**

    - Collective booking functionality for student groups
    - Group leader designation and responsibility assignment
    - Coordinated scheduling and confirmation processes

2. **Booking Cancellation System**

    - Individual student cancellation from group bookings
    - Complete group booking cancellation by group leader
    - Automatic capacity adjustment upon cancellation
    - Waitlist promotion when slots become available

3. **Capacity Management System**

    - Real-time slot availability monitoring
    - Automatic rejection of bookings exceeding capacity limits
    - Dynamic capacity updates based on cancellations
    - Waitlist management with automatic promotion

4. **Administrative Oversight**

    - Registration and cancellation tracking
    - Booking lifecycle management
    - System usage analytics and reporting

Business Rules and Constraints

- Each workshop slot maintains a maximum capacity threshold
- Group bookings are processed atomically (all-or-nothing basis)
- System prevents double-booking through database-level constraints
- Booking status must be tracked throughout the entire lifecycle
- Cancellations must be processed within specified timeframes
- Cancelled slots automatically become available for new bookings
- Waitlisted students are automatically promoted when slots open
- All workshops are provided free of charge (no payment processing required)

Core Assignment Requirements

Part 1A: Database Design and Implementation (CLO 2) - 2 Marks

Entity Architecture

Students must identify and model the following core entities:

Primary Entities:

- **Students**: Individual participant profiles and academic information
- **Groups**: Group formation and member management
- **Workshops**: Course offerings and specifications
- **Time Slots**: Scheduled workshop sessions with capacity limits
- **Bookings**: Reservation records and status tracking
- **Instructors**: Workshop facilitators and their assignments
- **Facilities**: Room allocation and resource management

You are tasked with designing a relational database for a workshop booking system. Based on the following primary entities, describe the relationships that would exist between them, including the type of relationship (e.g., one-to-many, many-to-many) and the implications for how they would be linked in a database schema. You should consider how each entity interacts with the others to ensure accurate data storage and retrieval.

Part 1B: Quality Assessment - Normalization Analysis (CLO 3) - 2 Marks

Instructions for Students:

You are tasked with designing and normalizing a relational database for a Workshop Booking System. Please follow these steps:

1. **Entity-Relationship Diagram (ERD) Generation:**
    - Based on the primary entities provided (Students, Groups, Workshops, Time Slots, Bookings, Instructors, Facilities), create a complete Entity-Relationship Diagram (ERD).
    - Your ERD should clearly show all entities, their attributes (including primary keys), and the relationships between them (including cardinality and participation).
2. **Normalization Assessment Questions:**
    - After generating your ERD, analyze your complete Workshop Booking System database schema for normalization compliance by answering the following questions. Provide detailed responses, supported by examples and clear justifications from your schema.

Normalization Assessment Questions:

- **Third Normal Form (3NF) Verification:**
    - Examine your database tables for any transitive dependencies.
    - Identify any non-key attributes that depend on other non-key attributes rather than directly on the primary key.
    - Provide specific examples from your schema showing functional dependency chains ($A \rightarrow B \rightarrow C$) and explain how you would resolve them to achieve 3NF if they exist.

Part 1C: Analytical Queries (CLO 4) - 2 Marks

Core Analytical Requirements

Students must develop SQL queries addressing:

Capacity Analysis:

- Weekly slot availability across all workshop categories
- Current utilization rates and booking patterns
- Available slots remaining for current week
- Total slots occupied for current week

Administrative Reporting:

- Group registration tracking and status monitoring
- Popular workshop identification and demand analysis
- Peak usage time identification

Operational Queries:

- Real-time capacity checking before booking acceptance
- Booking history and trends analysis
- Student participation tracking

Part 2: Team Report & Presentation (CLO 6) - 4 Marks

Submit a well-structured report covering design decisions, testing evidence, and reflection. The report should comprehensively document your Workshop Booking System database assignment, demonstrating your understanding of database principles and implementation processes.

Report Structure and Content:

Write a comprehensive report of at least 10 pages that covers all aspects of your Workshop Booking System assignment, including:

- Database design decisions and methodology
- Implementation approach and technical choices
- Testing strategies and validation evidence
- Query development and performance analysis
- Normalization analysis and quality assessment
- Challenges encountered and solutions implemented
- Team collaboration and assignment management
- Critical reflection on learning outcomes
- Future improvements and system enhancements

Part 3: Q&A Session (CLO 6) - 5 Marks

In this part, all the team members will participate in the **Question and Answer (Q&A) session** to assess their understanding and contributions to the Workshop Booking System assignment.

During the session, every student will be asked specific questions related to their assigned tasks, technical decisions, and problem-solving approaches. The purpose of this assessment is to evaluate each member's individual learning outcomes and their level of involvement in the assignment.

The Q&A session will cover the following aspects:

- Individual roles and responsibilities in the assignment.
- Understanding of database design and implementation.
- Explanation of key SQL queries developed by the student.
- Challenges faced and how they were resolved.
- Reflection on teamwork, communication, and contribution.

Each student will be marked based on their ability to clearly explain their work, demonstrate understanding of database concepts, and show evidence of their participation in the assignment.

Introduction

Many universities provide workshops for students to gain new skills and knowledge, however due to differences in each student's time slots, availability of rooms and capacity of students. Universities need to schedule and manage these workshops for different student groups. The Workshop booking system is a database management system which is designed to support this and help organize the workshop booking.

In this project, our team has created a database management system based on the system requirements, we have defined the entities needed for the booking process, designed the necessary relationships, normalization was applied to ensure our database avoids redundancy and maintains its integrity. MySQL was used to implement our database, we created the specific tables, we applied constraints, created the queries, triggers and procedures, to support the important system features.

This report explains the stages of the project which includes the designing, the implementation, testing strategies, challenges, improvement for the future. Finally, the purpose of our report is to display the concepts of database applied in building our workshop scheduling and management system.

System Overview:

The workshop booking system is created to help organize and manage the workshop reservations for the students. It can handle individual and group bookings; it ensures that rules such as the capacity rules are abided. Each workshop has several time slots with each having a limited amount of seats. Availability is checked by the system before allowing a booking to be made and prevents students from booking the same time slots twice.

Additionally, the workshop booking system automatically refreshes the availability of the seats and handles the waitlists and cancellations. The database contains and stores information about instructors, facilities, and reservations, enabling the administrators to monitor the usage and produce reports. The system offers a structured manner to store data and facilitate all booking activities.

Database Design Decisions & Methodology

The Design Approach:

The database model chosen was relational due to the data structure and the relations between the students , the instructors , the workshops and the facilities. To reduce and prevent data redundancy and maintain the integrity of the data , normalization was applied. Constraints and triggers were used to embed the business rules like the capacity limits , the responsibilities of the group leader , the lifecycle of the booking . Finally stored procedures were used to support the analytics and reporting.

Entities and their relationships

A number of entities form the core basis for the Workshop Booking System. The Students entity forms the representation of each individual participant, with stored academic details, contact information, and group membership. Any student is attached to one Group: a many-to-one relationship. Likewise, a student may have multiple Bookings, a one-to-many relationship. The Groups entity satisfies the overall structure of students, with the number of individuals and leaders of groups. Each group has one selected Leader, a one-to-one relationship, while a group can be associated with multiple Workshops, indicating the relationship between groups and their scheduling purposes.

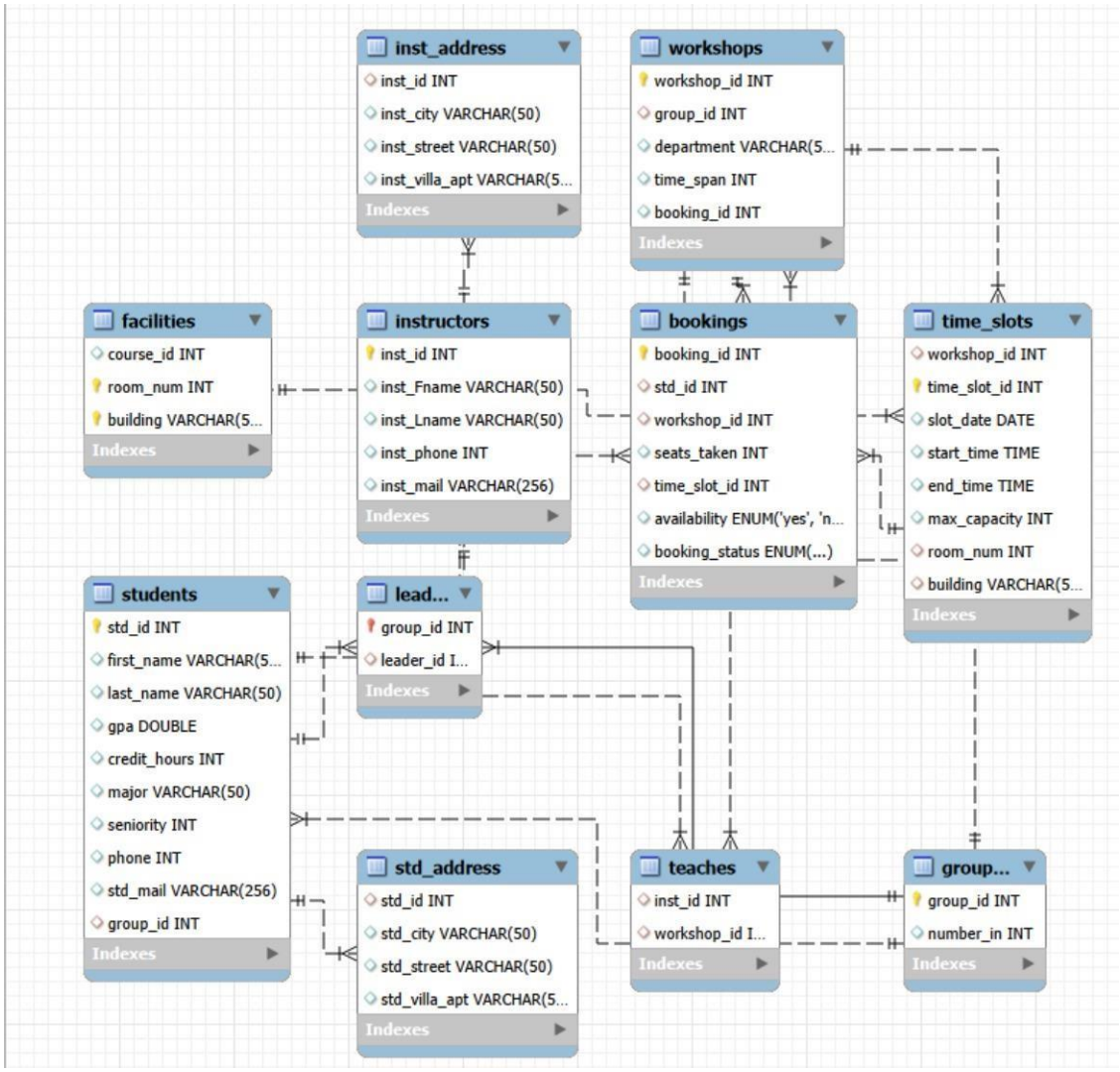
The Workshops entity signifies the course offerings and the attributes attached to the department, duration, and linked group. For every workshop, there can be multiple Time Slots in which sessions can be scheduled for the workshops-a one-to-many relationship. Equally, there are several Bookings for every workshop, showing reservations by students, and workshops can have several Instructors, showing a many-to-many relationship that is achieved through the Teaches junction table. The Time Slots entity captures the schedule details for each workshop session, including date, start and end times, maximum capacity, and room allocation.

The Bookings entity represents an individual or group booking, recording the student, workshop, time slot, number of seats taken, and the availability and status of bookings. It is related to Students, Workshops, and Time Slots on a many-to-one basis. The Instructors entity stores the information about facilitators; it is related to workshops through the Teaches table, which allows many-to-many relationships with them. Facilities, through the Facilities entity, maintain information about rooms and buildings.

Each facility has one-to-many relationships with the time slots in order to ensure that the rooms are appropriately allocated. Finally, the addresses for students and instructors will be in separate Address tables to maintain normalization. Each will be related to their respective primary entities. These entities and their relationships collectively describe the Workshop Booking System, allowing for the storing of data accurately, enforcing business rules, and also performing query operations efficiently.

The ERD for the Workshop Booking System graphically depicts all core entities, their attributes, and the relationships between them. Each entity, such as Students, Groups, Workshops, Time Slots, Bookings, Instructors, and Facilities, is represented along with the primary key and essential attributes. The diagram also illustrates cardinality and the participation of relationships, including the one-to-many between Students and Bookings; the many-to-many between Workshops and Instructors, which is implemented via the Teaches junction table; and the one-to-many between Facilities and Time Slots. This ERD acts as a blueprint for the database schema to ensure that data storage is well organized, constraints are duly enforced, and all business rules are clearly modeled.

Figure: Workshop Booking System ERD



Implementation Approach and Technical Choices

The implementation of the Workshop Booking System was done using MySQL Workbench. The design of the database schema was thus done based on the ER diagram done earlier, making sure that all the entities, attributes, and relationships were correctly implemented. Each table was designed to have appropriate primary keys to uniquely identify the records, and foreign keys, so that relationships between tables are always maintained with referential integrity.

UNIQUE, NOT NULL, and FOREIGN KEY constraints were applied to ensure no invalid data would be present and to implement business rules. For example, the combination of std_id and time_slot_id was made unique in the Bookings table, to avoid an instance where one student can book the same time slot multiple times.

The system also consists of triggers and stored procedures in handling dynamic rules to facilitate reporting. A trigger was implemented that did not allow overbooking a time slot beyond its maximum capacity. Stored procedures were implemented for operational and

analytical queries on student booking history, overall booking trends, peak usage time slots, workshop demand analysis, and student participation.

The technical choices were governed by the following needs:

Data Integrity Using Constraints and Normalization.

Automation: use of triggers in capacity control and waitlist promotion.

Efficiency: stored procedures for fast execution of repetitive analytical queries.

Scalability: A normalized design allows for many-to-many relationships using junction tables, making it easily scalable for additional workshops, instructors, and facilities.

Evidence of the implementation:

- Database Creation: creates the database of the project and sets it as the database for all the table creations and the queries.

CREATE DATABASE project;

USE project;

- Create Tables:
- The Student's Table: This table stores all the student records and information.

```
CREATE TABLE students (
    std_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    gpa DOUBLE,
    credit_hours INT,
    major VARCHAR(50),
    seniority INT,
    phone INT,
    std_mail VARCHAR(256),
    group_id INT
);
```

- The Booking table with its UNIQUE constraints: tracks and stores all the bookings for the workshops.

```
CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    std_id INT,
    workshop_id INT,
    seats_taken INT,
    time_slot_id INT,
    availability ENUM('yes', 'no'),
    booking_status ENUM('pending','confirmed','cancelled','waitlisted'),
    FOREIGN KEY (std_id) REFERENCES students(std_id),
```

```
    FOREIGN KEY (workshop_id) REFERENCES workshops(workshop_id),
    FOREIGN KEY (time_slot_id) REFERENCES time_slots(time_slot_id),
    CONSTRAINT uniq_student_slot UNIQUE(std_id, time_slot_id)
);
```

- Trigger: it will prevent overbooking by checking if a time slot's total bookings are exceeding 40 before it inserts a new booking , if this happens it will stop the insertions and raise errors.

```
DELIMITER //
CREATE TRIGGER check_before_insert
BEFORE INSERT ON bookings
FOR EACH ROW
BEGIN
    IF (SELECT COUNT(*)
        FROM bookings
        WHERE time_slot_id = NEW.time_slot_id) >= 40 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Maximum of 40 students reached for this time
slot.';
    END IF;
END;//
DELIMITER ;
```

- Stored Procedure:
- Student history: gets all the records of the bookings for a specific student based on their student ID which allows the administrators to see a student's booking history.

```
DELIMITER //
CREATE PROCEDURE get_student_history(IN p_std_id INT)
BEGIN
    SELECT booking_id, workshop_id, time_slot_id, seats_taken,
booking_status, availability
    FROM bookings
    WHERE std_id = p_std_id;
END //
DELIMITER ;
```

- The Workshop Demand Analysis; Displays the analytic insights of each workshop with the total bookings , total seats which are taken and the average number of seats per booking.

```
DELIMITER //
CREATE PROCEDURE get_demand_analysis()
BEGIN
    SELECT workshop_id, COUNT(*) AS num_bookings, SUM(seats_taken) AS
total_seats,
           AVG(seats_taken) AS avg_seats_per_booking
    FROM bookings
    GROUP BY workshop_id
```

```
    ORDER BY total_seats DESC;
END //
DELIMITER ;
```

Testing Strategies and Validation Evidence

Structural testing was conducted on the Workshop Booking System to ensure that each component performed its intended function and satisfied all the requirements of the project. Testing first started with structural validation, in which each table was checked to confirm that the correct data type, primary key, and foreign key were applied.

A number of test insertions were tried to confirm that constraints on NOT NULL, UNIQUE, and FOREIGN KEY worked as they were supposed to. For example, efforts to insert duplicate bookings for the same time slot and student were rejected, as expected, indicating that a uniqueness rule had been implemented appropriately. Functional testing was conducted next to validate that the system would act accordingly for any given scenario.

The trigger that prevented overbooking was tested by continually inserting booking entries into one time slot until its capacity was reached. After the limit had been surpassed, the system correctly prevented further bookings and provided a proper error message, ensuring that capacity control was enforced. Testing stored procedures, such as those responsible for retrieving the history of bookings and trends of workshop demand, involved the use of different inputs to ensure that correct and complete results were returned. The testing of relationships also verified that it was impossible to create invalid or orphan records, meaning data accuracy and coherence were maintained throughout.

Query Development and Performance Analysis

The booking system required both operational and analytical support; a set of various queries was developed for checking available time slots, popular workshops, viewing booking trends, and tracking student participation. These queries had been designed to be efficient using proper joins, grouping operations, and conditions that minimized unnecessary processing. It was also observed during testing that the normalized schema contributed much to the efficiency of queries because related data is well organized and easy to retrieve.

The performance analysis indicated that most of the queries ran quick because of the use of primary keys and foreign keys on a well-structured design. Even when combining tables, the data processed was fast for some analytical queries, like those used to calculate utilization rates or list remaining slots. For further improvements in performance, the use of stored procedures minimized the processing of repetitive queries and enabled faster execution of commonly used operations. Overall, the system demonstrated strong performance in handling both basic and complex queries.

Normalization Analysis and Quality Assessment

The database design was checked for complete normalization, ensuring that it met the First, Second, and Third Normal Form. All tables conformed to 1NF because atomic values in each field did not have any repeating groups. It further met 2NF since every non-key attribute was

fully dependent on its table's primary key, and no partial dependencies existed. Finally, 3NF was achieved because no non-key attribute depended upon another non-key attribute; instead, every attribute completely depended on the primary key.

For the enforcement of 3NF , we added primary keys to several of the tables that previously did not have one :

```
ALTER TABLE std_address ADD PRIMARY KEY (std_id, std_city, std_street, std_villa_apt);

ALTER TABLE inst_address ADD PRIMARY KEY (inst_id, inst_city, inst_street, inst_villa_apt);

ALTER TABLE teaches ADD PRIMARY KEY (inst_id, workshop_id);
```

The introduction of these primary keys ensured that the tables did not rely on non-key attributes, thus eliminating any transitive dependencies. Now each attribute is dependent only on the primary key, hence ensuring data integrity and eliminating redundancy. Such normalization eliminated data redundancy, avoided any anomalies such as update and deletion, and enhanced consistency across all aspects of the database. By putting students, instructors, workshops, and time slots in different tables, there will be no unnecessary repetition of information. Using junction tables for instance, the table connecting instructors to workshops has correctly represented many-to-many relationships. Thus, this database design is efficient, well-organized, and can very well support future scaling.

Challenges encountered and solutions implemented

In developing the system, a number of issues arose. One was that tables had to be created in order, due to the foreign key dependencies between some tables. This was overcome by planning out the order in which tables would be created and ensuring that any parent tables were created before the child tables referencing them.

Another challenge arose in capacity control using triggers. The first logic of the trigger did not correctly flag when the maximum capacity in a given timeslot was reached. Later, after considering carefully revised conditions and testing different cases, it was modified to correctly flag the instance when the capacity is reached and does not allow further bookings.

Stored procedures were also initially problematic, especially when designing analytical queries that combined multiple tables. Grouping, filtering, or aliasing errors were updated by refining query structures and conducting exhaustive testing. With this, the team further enhanced its SQL programming logic, debugging strategies, and other database constraints.

Team Collaboration and assignment management

The work was divided between the three team members. Designing the ERD and establishing the database schema were assigned to one member, while others did table creation, constraints, stored procedures, triggers, and documentation. Regular communication supported the consistency of the design in all parts. Shared files and incremental testing prevented conflicts or errors related to outdated versions. Collaboration also played an important role in solving problems. Team members supported each other with the code, reviewing logic, and enhancing design decisions. This collective effort helped to bring the system to a successful completion.

Critical reflection on learning outcomes

This project involved applying theoretical concepts in developing a practical database system. Through the process of designing ERD, creating tables, and writing SQL queries, we came to understand more thoroughly relational structures, normalization, and referential integrity. Trigger building and stored procedures introduced us to advanced SQL functionalities by showing us how business rules could be automated at the database level. Testing and debugging also furthered this message about detailed work and iterative improvement. Overall, this group project further developed our technical competencies, critical thinking, and confidence in using functional database systems.

Future improvements and system enhancements

Although the current system is sufficient to meet all core needs, several improvements can enhance its functionality and usability even further. For instance, improving the current system with a graphical user interface will help students and administrators communicate easily with the system instead of depending on SQL commands. It would be more effective and friendly for users if automated email notifications were developed, along with waitlist handling and real-time updates of available seats. Such amendments could be integrated into the system. Additionally, indexing frequently searched fields in queries can enhance query speeds once the database grows larger. Further versions could include more advanced analytics, like predicting workshop demand or recommending suitable sessions for particular students. These enhancements would create a more robust, scalable, and adaptable system that meets real-world institutional needs.

Source Code:

```
use project;

create table students (

std_id int primary key auto_increment,

first_name varchar(50),

last_name varchar(50),

gpa double,

credit_hours int,

major varchar(50),

seniority int,

phone int,

std_mail varchar(256),

group_id int

);
```

```
create table std_address (
std_id int,
std_city varchar(50),
std_street varchar(50),
std_villa_apt varchar(50),
foreign key (std_id) references students (std_id)
);

create table Groups_ (
group_id  int primary key,
number_in int
);

create table leader (
group_id  int primary key,
leader_id int unique,
foreign key (group_id) references Groups_ (group_id),
foreign key (leader_id) references students (std_id)
);

create table workshops (
workshop_id int primary key,
department varchar(50),
time_span int
);

create table time_slots (
workshop_id  int,
time_slot_id int primary key auto_increment,
slot_date date,
start_time time,
end_time time,
```

```
max_capacity int,
room_num int,
building varchar(50),
foreign key (workshop_id) references workshops (workshop_id)
);

create table bookings (
booking_id int auto_increment primary key,
std_id int,
workshop_id int,
seats_taken  int,
time_slot_id int,
availability enum ('yes', 'no'),
booking_status enum('pending','confirmed','cancelled','waitlisted'),
foreign key (std_id) references students (std_id),
foreign key (workshop_id)  references workshops (workshop_id),
foreign key (time_slot_id) references time_slots (time_slot_id),
constraint uniq_student_slot unique (std_id, time_slot_id)
);

create table instructors (
inst_id int primary key,
inst_Fname varchar(50),
inst_Lname varchar(50),
inst_phone int,
inst_mail  varchar(256)
);

create table teaches (
inst_id int,
workshop_id int,
foreign key (inst_id) references instructors (inst_id),
```

```
foreign key (workshop_id) references workshops (workshop_id)
);

create table inst_address (
inst_id int,
inst_city varchar(50),
inst_street varchar(50),
inst_villa_apt varchar(50),
foreign key (inst_id) references instructors (inst_id)
);

create table facilities (
workshop_id int,
room_num int,
building varchar(50),
primary key (room_num, building)
);

ALTER TABLE facilities ADD CONSTRAINT workshop FOREIGN KEY (workshop_id) REFERENCES workshops(workshop_id);
alter table students add constraint students_group foreign key (group_id) references Groups_ (group_id);
alter table time_slots add constraint location foreign key (room_num, building) references facilities (room_num, building);
alter table std_address add primary key (std_id, std_city, std_street, std_villa_apt);
alter table inst_address add primary key (inst_id, inst_city, inst_street, inst_villa_apt);
alter table teaches add primary key (inst_id, workshop_id);

delimiter //
create procedure get_famous()
begin
    select workshop_id, sum(seats_taken) as total_seats
```

```
    from bookings
    group by workshop_id
    having sum(seats_taken) > 30;
end;//
delimiter ;
call get_famous();



delimiter //
create trigger check_before_insert
before insert on bookings       for each row
begin
    if (select count(*)
        from bookings
        where time_slot_id = new.time_slot_id) >= 40 then

        signal sqlstate '45000'
            set message_text = 'Maximum of 40 students reached for this time slot.';
    end if;
end;//
delimiter ;

delimiter //
create procedure get_student_history(in p_std_id int)
begin
select booking_id, workshop_id, time_slot_id, seats_taken, booking_status, availability
from bookings where std_id = p_std_id;
end //
delimiter ;
call get_student_history(103);

delimiter //
```

```
create procedure get_booking_history()
begin
   select booking_id, std_id, workshop_id, time_slot_id, seats_taken, booking_status, availability
   from bookings order by booking_id;
end //
delimiter ;
call get_booking_history();


delimiter //
create procedure get_reg_trackk(in p_group_id int)
begin
select std.group_id,std.std_id,std.first_name,std.last_name,
bkng.booking_id,bkng.workshop_id,bkng.time_slot_id,
bkng.seats_taken,bkng.booking_status,bkng.availability
from students std left join bookings bkng on std.std_id = bkng.std_id
where std.group_id = p_group_id order by std.std_id, bkng.booking_id;
end //
delimiter ;
call get_reg_trackk(4);


delimiter //
create procedure get_demand_analysis()
begin
select
        workshop_id,
        count(*)        as num_bookings,
        sum(seats_taken) as total_seats,
        avg(seats_taken) as avg_seats_per_booking
from bookings group by workshop_id order by total_seats desc;
end //
delimiter ;
call get_demand_analysis();
```

```
delimiter //
create procedure get_booking_trends()
begin
select booking_status, count(*) as num_bookings, sum(seats_taken) as total_seats
from bookings group by booking_status;
end //
delimiter ;
call get_booking_trends();

delimiter //
create procedure get_peak_usage()
begin
select time_slot_id, sum(seats_taken) as total_seats
from bookings group by time_slot_id order by total_seats desc;
end //
delimiter ;
call get_peak_usage();

delimiter //
create procedure get_student_participationn()
begin
select std_id, count(*) as num_bookings
from bookings group by std_id order by num_bookings desc;
end //
delimiter ;
call get_student_participationn();

delimiter //
create procedure slots_avlb()
begin
```

```
SELECT w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id, slot.room_num, slot.building, (slot.max_capacity - SUM(b.seats_taken))

AS avlb_seats FROM workshops w JOIN time_slots slot ON w.workshop_id=slot.workshop_id

LEFT JOIN bookings b ON b.time_slot_id=slot.time_slot_id group by slot.time_slot_id HAVING avlb_seats > 0 ORDER BY slot.slot_date ASC;

end //

delimiter ;

call slots_avlb();


delimiter //

create procedure slots_taken()

begin

select w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id, slot.room_num, slot.building,

(sum(b.seats_taken), 0) as seats_taken

from workshops w

join time_slots slot on w.workshop_id = slot.workshop_id

left join bookings b on b.time_slot_id = slot.time_slot_id

group by w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id, slot.room_num, slot.building order by slot.slot_date asc;

end //

delimiter ;

call slots_taken();


delimiter //

create procedure booking_rate()

begin

select w.workshop_id, w.department, slot.slot_date, slot.max_capacity, sum(b.seats_taken) as booked, round((IFNULL(SUM(b.seats_taken),0)/slot.max_capacity)*100,0) AS booking_percent

from workshops w join time_slots slot on w.workshop_id = slot.workshop_id left join bookings b on b.time_slot_id = slot.time_slot_id

group by w.workshop_id, w.department, slot.slot_date, slot.max_capacity order by slot.slot_date asc;
```

```
end //
delimiter ;
call booking_rate();
```

TABLES OUTPUT:

EVIDENCE 1 :

```
1 •  SELECT * FROM project.students;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

std_id	first_name	last_name	gpa	credit_hours	major	seniority	phone	std_mail	group_id
101	aysha	ejaz	4	12	cyber	3	56336	as@ma...	1
102	leanne	rodrigo	4	15	cs	4	42135	lr@mail...	2
103	yasmin	issa	4	16	cyber	2	58643	yai@m...	4
104	alia	ahmad	4	10	cs	3	55379	awd@...	3
105	asma	lin	4	11	cs	2	59383	asm@...	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EVIDENCE 2 :

```
1 •  SELECT * FROM project.groups_;
```

Result Grid | Filter Rows: | Edit:

group_id	number_in
1	2
2	2
3	4
4	5
NULL	NULL

EVIDENCE 3 :

```
1 •  SELECT * FROM project.facilities;
```

Result Grid | Filter Rows: | Edit: | Export/I

workshop_id	room_num	building
301	3	hub
302	2	mgt
303	8	mgt
NULL	NULL	NULL

EVIDENCE 4:

```
1 •  SELECT * FROM project.facilities;
```

Result Grid | Filter Rows: | Edit: | Export/I

workshop_id	room_num	building
301	3	hub
302	2	mgt
303	8	mgt
NULL	NULL	NULL

EVIDENCE 5 :

```
1 •  SELECT * FROM project.time_slots;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

workshop_id	time_slot_id	slot_date	start_time	end_time	max_capacity	room_num	building
301	1	2025-11...	14:00:00	17:00:00	40	3	hub
302	2	2025-11...	09:00:00	12:00:00	40	2	mgt
303	3	2025-11...	16:00:00	18:00:00	40	8	mgt
302	4	2025-12...	12:00:00	15:00:00	40	2	mgt
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EVIDENCE 6 :

```
1 •  SELECT * FROM project.bookings;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

booking_id	std_id	workshop_id	seats_taken	time_slot_id	availability	booking_status
1	101	301	39	1	yes	confirmed
2	102	302	18	2	yes	pending
3	103	301	0	1	no	cancelled
4	104	302	20	4	yes	confirmed
5	105	303	11	3	yes	confirmed
6	103	303	8	3	yes	confirmed
NULL	NULL	NULL	NULL	NULL	NULL	NULL

EVIDENCE 7 :

```
1  SELECT * from project.leaders;
```

Result Grid | Filter Rows: | Edit: | Export/Im

group_id	leader_id
1	101
2	102
3	104
4	105
NULL	NULL

EVIDENCE 8 :

```
1 •  SELECT * FROM project.teaches;
```

Result Grid | Filter Rows: | Edit:

inst_id	workshop_id
300	301
310	302
320	303
NULL	NULL

EVIDENCE 9 :

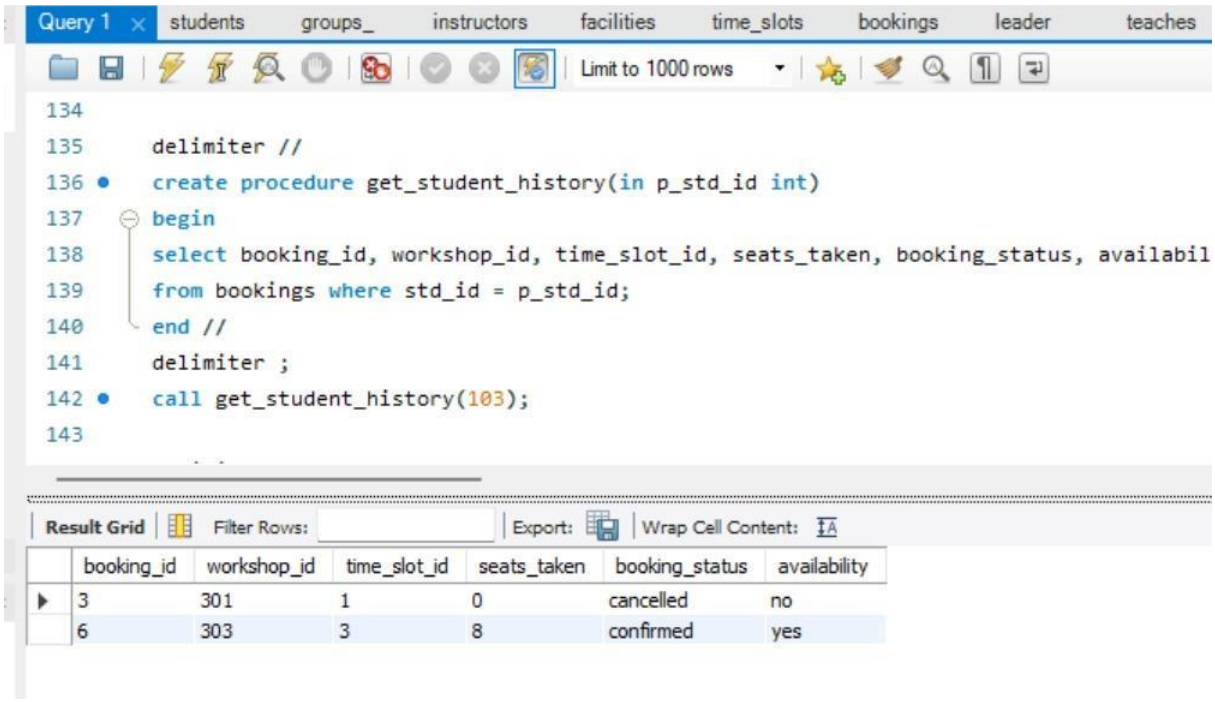
```
1 •  SELECT * FROM project.workshops;
```

Result Grid | Filter Rows: | Edit: | Export/In

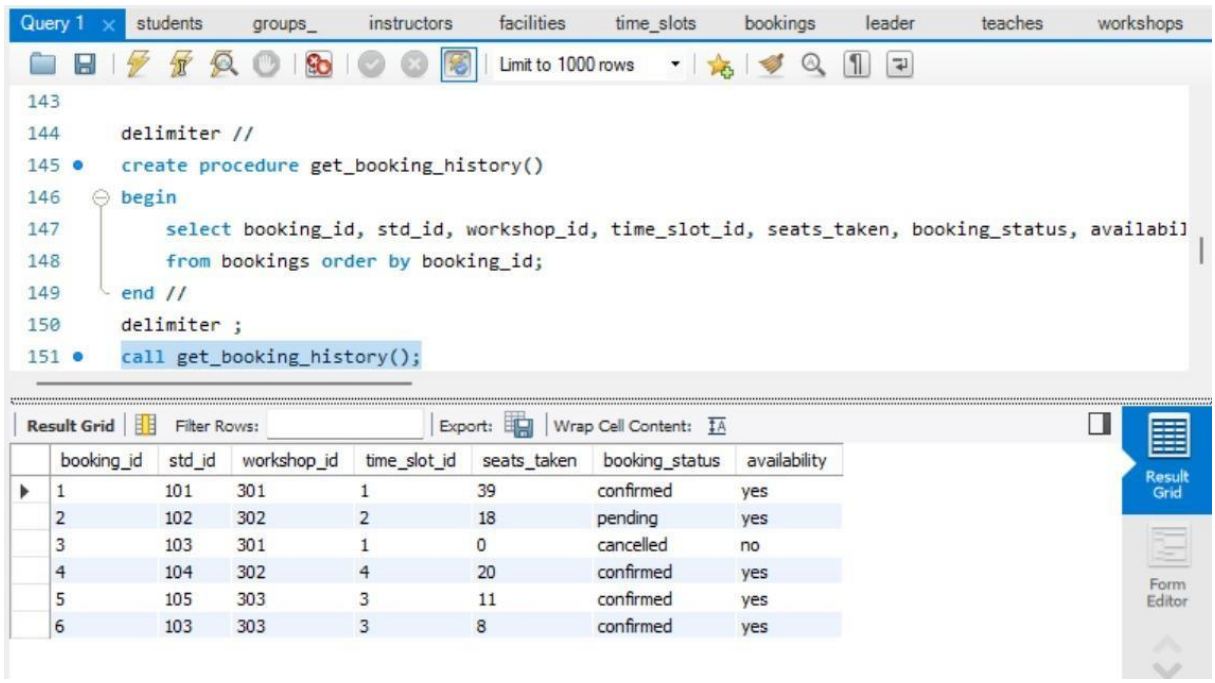
workshop_id	department	time_span
301	ai	3
302	robotics	3
303	ent	3
NULL	NULL	NULL

PROCEDURES OUTPUT :

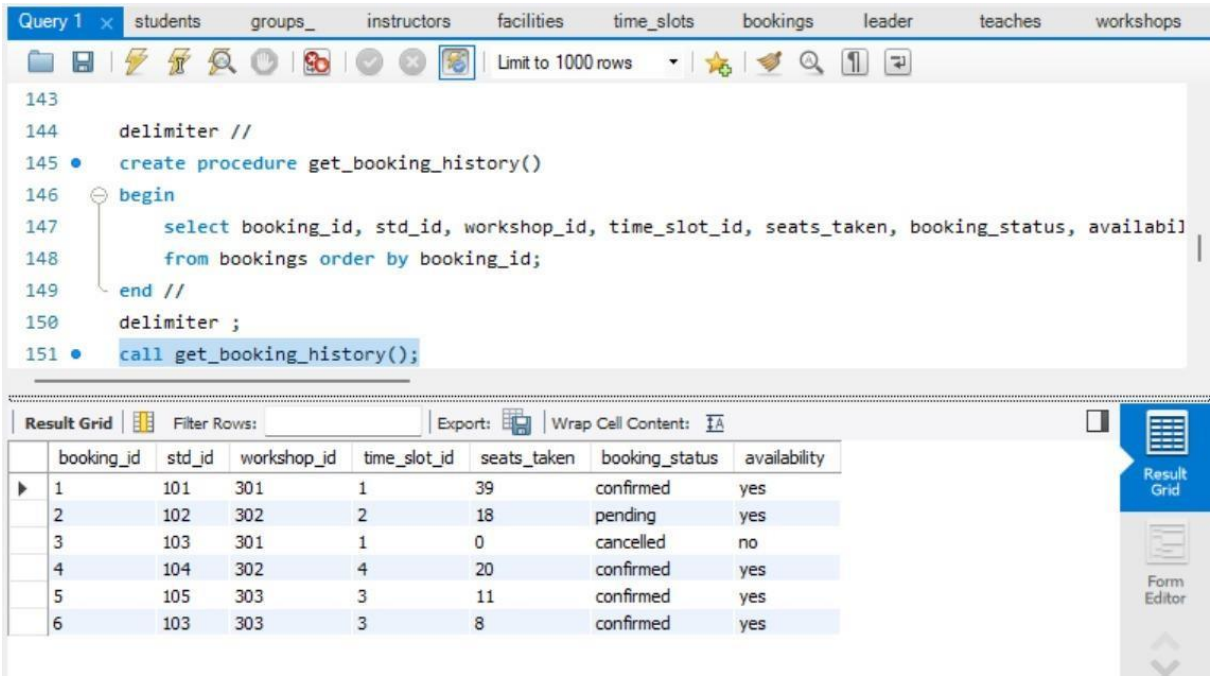
EVIDENCE 1 :



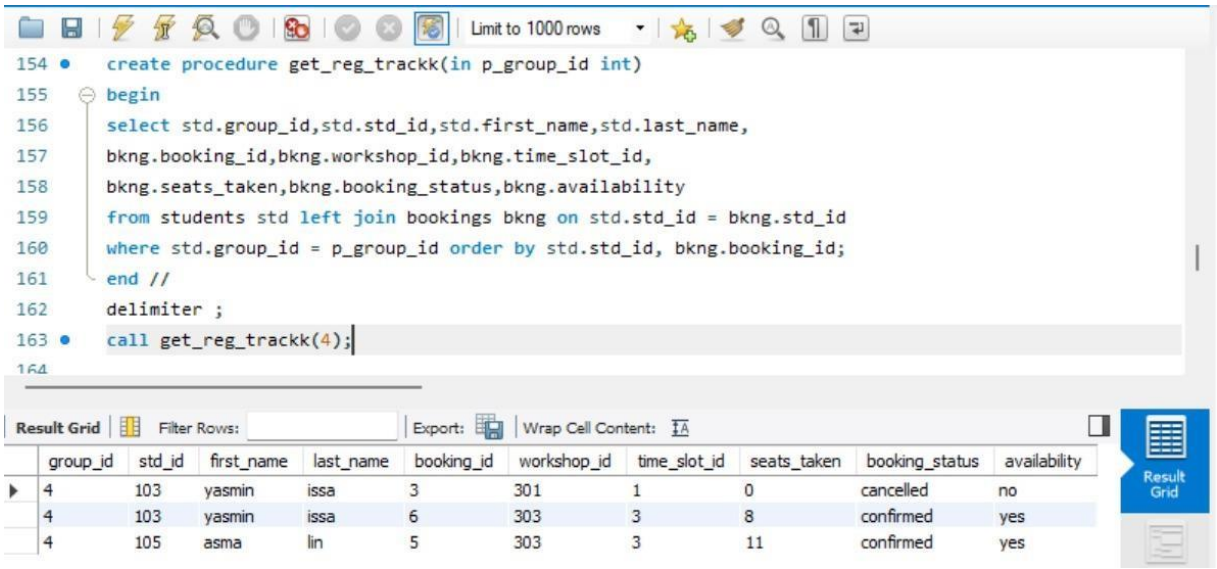
EVIDENCE 2 :



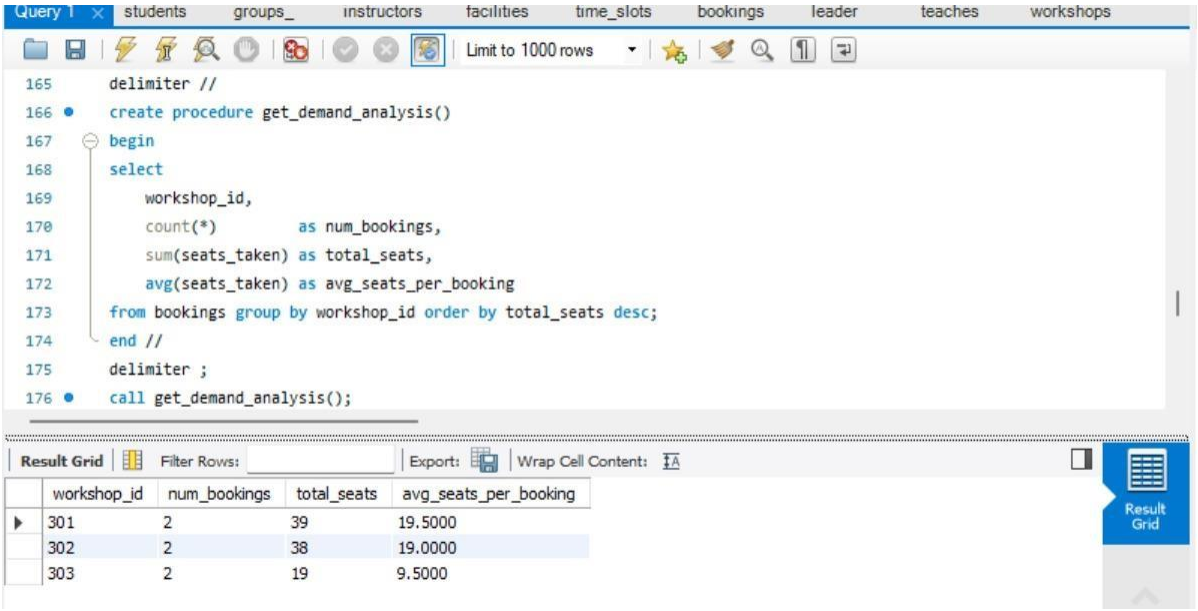
EVIDENCE 3 :



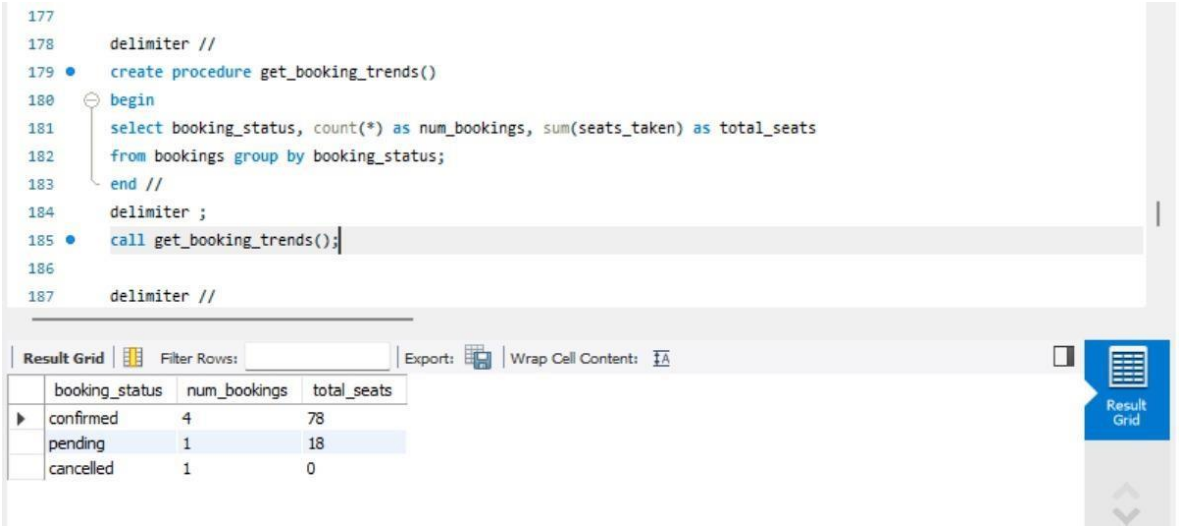
EVIDENCE 4 :



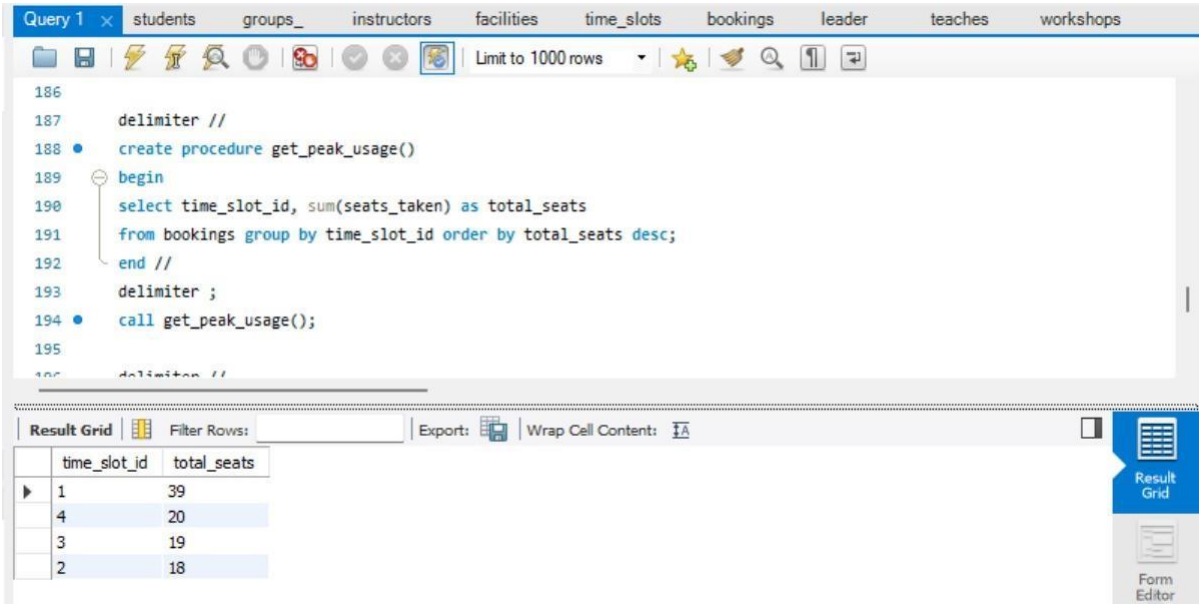
EVIDENCE 5 :



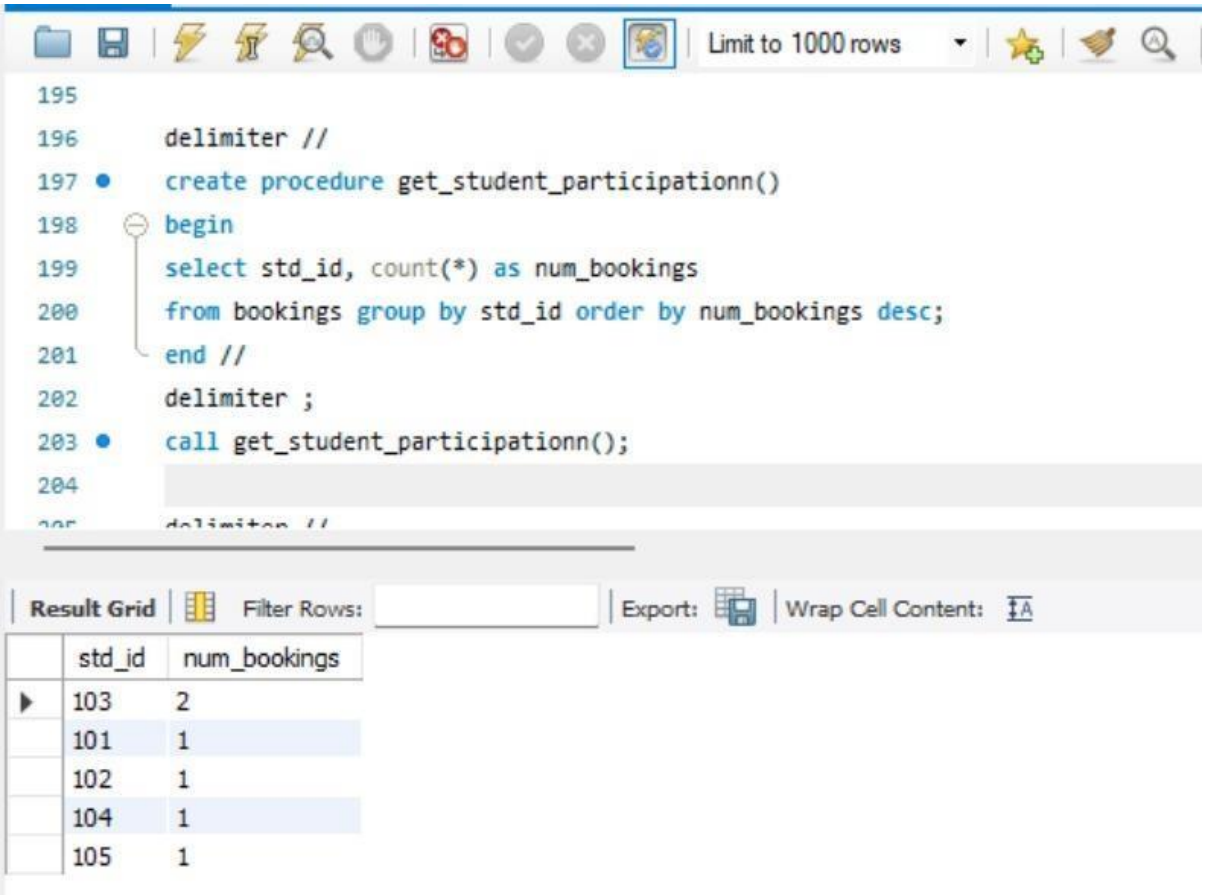
EVIDENCE 6 :



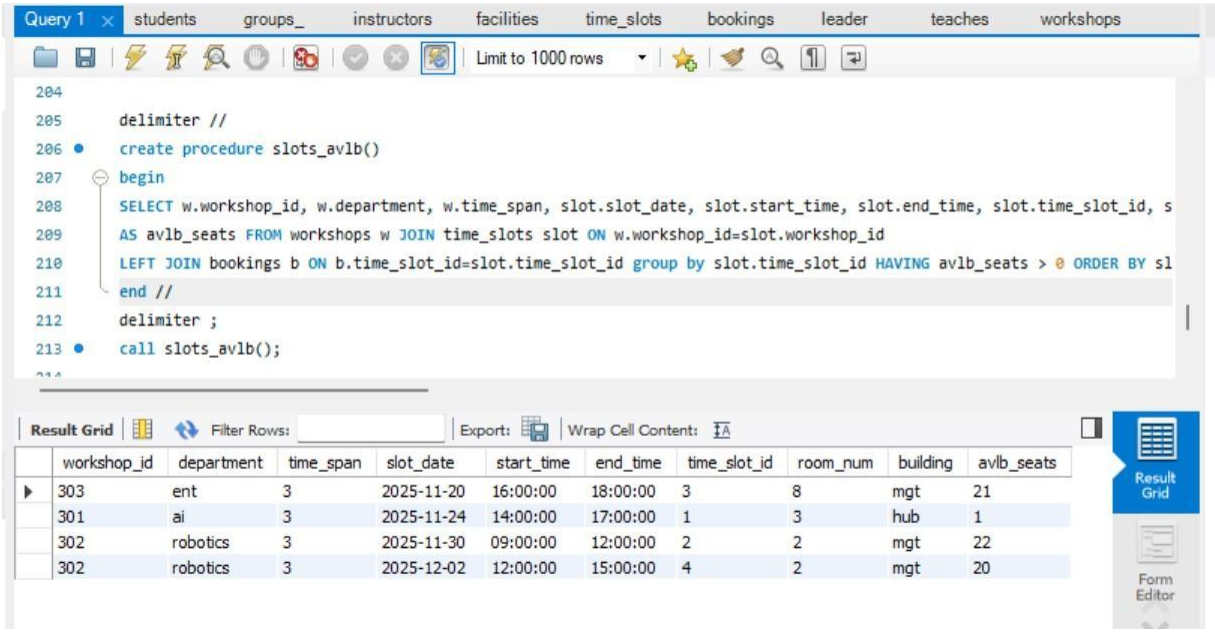
EVIDENCE 7:



EVIDENCE 8 :



EVIDENCE 9 :



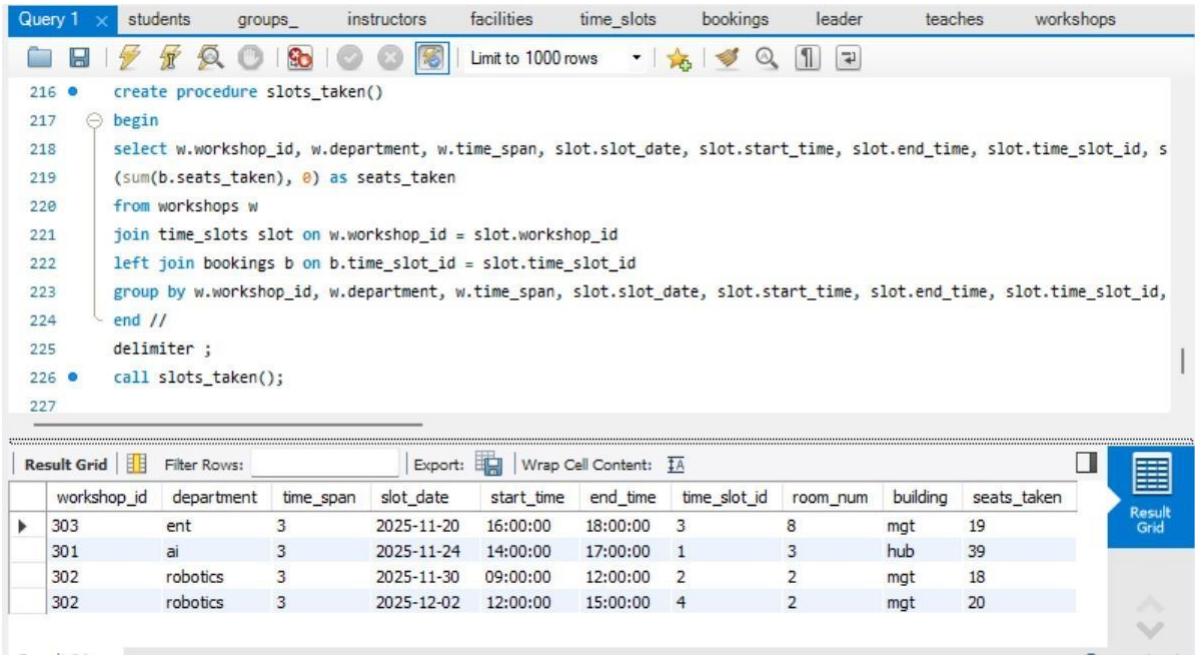
```
Query 1 | students | groups_ | instructors | facilities | time_slots | bookings | leader | teaches | workshops
Limit to 1000 rows

204
205     delimiter //
206 •   create procedure slots_avlb()
207     begin
208     SELECT w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id, s
209     AS avlb_seats FROM workshops w JOIN time_slots slot ON w.workshop_id=slot.workshop_id
210     LEFT JOIN bookings b ON b.time_slot_id=slot.time_slot_id group by slot.time_slot_id HAVING avlb_seats > 0 ORDER BY sl
211     end //
212     delimiter ;
213 •   call slots_avlb();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

workshop_id	department	time_span	slot_date	start_time	end_time	time_slot_id	room_num	building	avlb_seats
303	ent	3	2025-11-20	16:00:00	18:00:00	3	8	mgt	21
301	ai	3	2025-11-24	14:00:00	17:00:00	1	3	hub	1
302	robotics	3	2025-11-30	09:00:00	12:00:00	2	2	mgt	22
302	robotics	3	2025-12-02	12:00:00	15:00:00	4	2	mgt	20

Result Grid | Form Editor

EVIDENCE 10 :



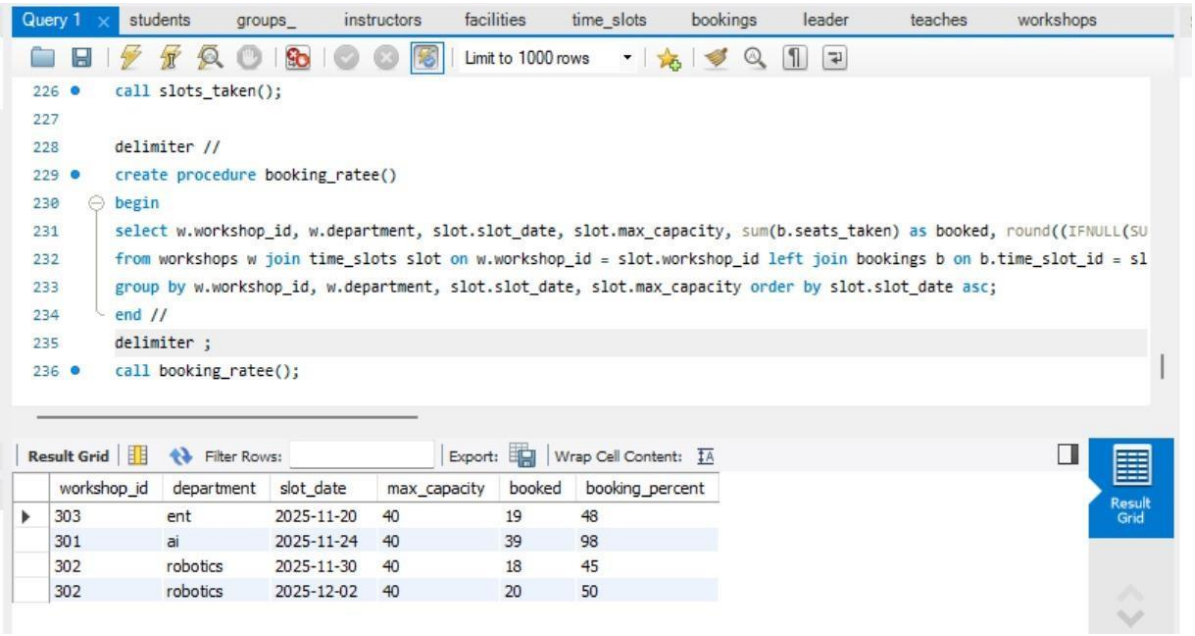
```
Query 1 | students | groups_ | instructors | facilities | time_slots | bookings | leader | teaches | workshops
Limit to 1000 rows

216 •   create procedure slots_taken()
217     begin
218     select w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id, s
219     (sum(b.seats_taken), 0) as seats_taken
220     from workshops w
221     join time_slots slot on w.workshop_id = slot.workshop_id
222     left join bookings b on b.time_slot_id = slot.time_slot_id
223     group by w.workshop_id, w.department, w.time_span, slot.slot_date, slot.start_time, slot.end_time, slot.time_slot_id,
224     end //
225     delimiter ;
226 •   call slots_taken();
227
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

workshop_id	department	time_span	slot_date	start_time	end_time	time_slot_id	room_num	building	seats_taken
303	ent	3	2025-11-20	16:00:00	18:00:00	3	8	mgt	19
301	ai	3	2025-11-24	14:00:00	17:00:00	1	3	hub	39
302	robotics	3	2025-11-30	09:00:00	12:00:00	2	2	mgt	18
302	robotics	3	2025-12-02	12:00:00	15:00:00	4	2	mgt	20

Result Grid

EVIDENCE 11 :



workshop_id	department	slot_date	max_capacity	booked	booking_percent
303	ent	2025-11-20	40	19	48
301	ai	2025-11-24	40	39	98
302	robotics	2025-11-30	40	18	45
302	robotics	2025-12-02	40	20	50